Cryptography and Network Security

Fifth Edition
by William Stallings

Chapter3

Based on the lecture slides by Lawrie Brown for “Cryptography and Network Security”, 5/e, by William Stallings, Chapter 3 – “Block Ciphers and the Data Encryption Standard”.

Chapter 3 – Block Ciphers and the Data Encryption Standard

All the afternoon Mungo had been working on Stern's code, principally with the aid of the latest messages which he had copied down at the Nevin Square drop. Stern was very confident. He must be well aware London Central knew about that drop. It was obvious that they didn't care how often Mungo read their messages, so confident were they in the impenetrability of the code.

—***Talking to Strange Men*, Ruth Rendell**

Intro quote.

Modern Block Ciphers

- now look at modern block ciphers
- one of the most widely used types of cryptographic algorithms
- provide secrecy /authentication services
- focus on DES (Data Encryption Standard)
- to illustrate block cipher design principles

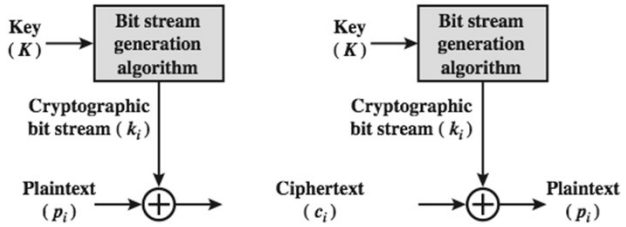
The objective of this chapter is to illustrate the principles of modern symmetric ciphers. For this purpose, we focus on the most widely used symmetric cipher: the Data Encryption Standard (DES). Although numerous symmetric ciphers have been developed since the introduction of DES, and although it is destined to be replaced by the Advanced Encryption Standard (AES), DES remains the most important such algorithm. Further, a detailed study of DES provides an understanding of the principles used in other symmetric ciphers. This chapter begins with a discussion of the general principles of symmetric block ciphers. Next, we cover full DES. Following this look at a specific algorithm, we return to a more general discussion of block cipher design.

Block vs Stream Ciphers

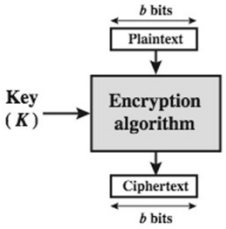
- block ciphers process messages in blocks, each of which is then en/decrypted
- like a substitution on very big characters
  - 64-bits or more
- stream ciphers process messages a bit or byte at a time when en/decrypting
- many current ciphers are block ciphers
  - better analysed
  - broader range of applications

Block ciphers work a on block / word at a time, which is some number of bits. All of these bits have to be available before the block can be processed. Stream ciphers work on a bit or byte of the message at a time, hence process it as a "stream". Block ciphers are currently better analysed, and seem to have a broader range of applications, hence focus on them.

Block vs Stream Ciphers

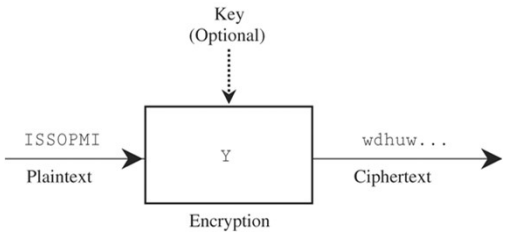


(a) Stream Cipher Using Algorithmic Bit Stream Generator

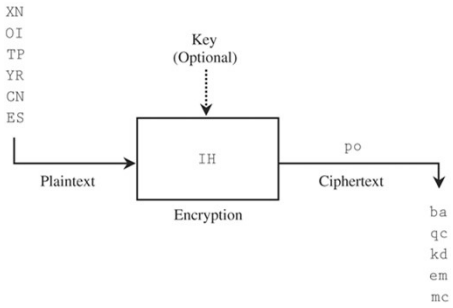


(b) Block Cipher

A block cipher is one in which a block of plaintext is treated as a whole and used to produce a ciphertext block of equal length. Typically, a block size of 64 or 128 bits is used. As with a stream cipher, the two users share a symmetric encryption key (Figure 3.1b). A stream cipher is one that encrypts a digital data stream one bit or one byte at a time. In the ideal case, a one-time pad version of the Vernam cipher would be used (Figure 2.7), in which the keystream (k) is as long as the plaintext bit stream (p).



Stream Encryption.



Block Cipher Systems.

Reversible/Irreversible mapping

N=2 → 2-bit input

Reversible Mapping	
Plaintext	**Ciphertext**
00	11
01	10
10	00
11	01

Irreversible Mapping	
Plaintext	**Ciphertext**
00	11
01	10
10	01
11	01

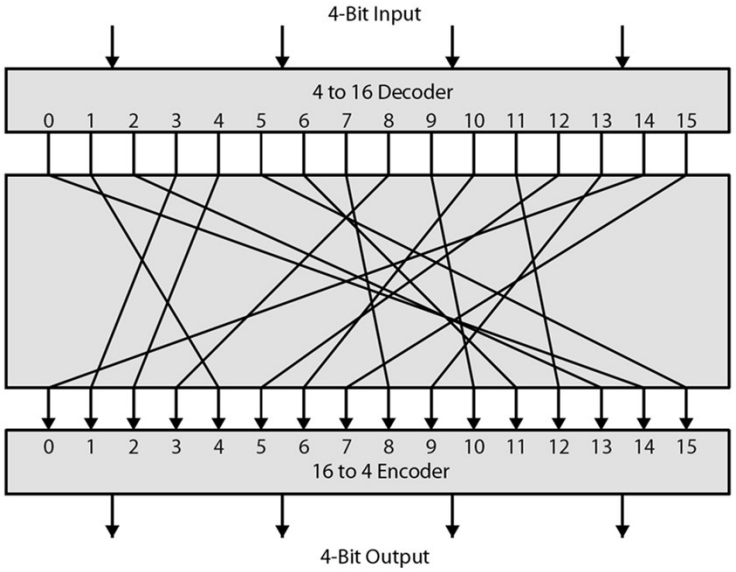
for A 4-bit input produces one of 16 possible input states,

Block Cipher Principles

- most symmetric block ciphers are based on a **Feistel Cipher Structure**
- needed since must be able to **decrypt** ciphertext to recover messages efficiently
- block ciphers look like an extremely large substitution
- would need table of 2^{64} entries for a 64-bit block
- instead create from smaller building blocks
- using idea of a product cipher

Most symmetric block encryption algorithms in current use are based on a structure referred to as a Feistel block cipher. A block cipher operates on a plaintext block of n bits to produce a ciphertext block of n bits. An arbitrary reversible substitution cipher for a large block size is not practical, however, from an implementation and performance point of view. In general, for an *n*-bit general substitution block cipher, the size of the key is $n \times 2^n$. For a 64-bit block, which is a desirable length to thwart statistical attacks, the key size is $64 \times 2^{64} = 2^{70} = 10^{21}$ bits. In considering these difficulties, Feistel points out that what is needed is an approximation to the ideal block cipher system for large n, built up out of components that are easily realizable.

Ideal Block Cipher



Feistel refers to an *n*-bit general substitution as an ideal block cipher, because it allows for the maximum number of possible encryption mappings from the plaintext to ciphertext block. A 4-bit input produces one of 16 possible input states, which is mapped by the substitution cipher into a unique one of 16 possible output states, each of which is represented by 4 ciphertext bits. The encryption and decryption mappings can be defined by a tabulation, as shown in Stallings Figure 3.2. It illustrates a tiny 4-bit substitution to show that each possible input can be arbitrarily mapped to any output - which is why its complexity grows so rapidly.

Mapping table

Table 3.1 Encryption and Decryption Tables for Substitution Cipher of Figure 3.2

Plaintext	Ciphertext
0000	1110
0001	0100
0010	1101
0011	0001
0100	0010
0101	1111
0110	1011
0111	1000
1000	0011
1001	1010
1010	0110
1011	1100
1100	0101
1101	1001
1110	0000
1111	0111

Ciphertext	Plaintext
0000	1110
0001	0011
0010	0100
0011	1000
0100	0001
0101	1100
0110	1010
0111	1111
1000	0111
1001	1101
1010	1001
1011	0110
1100	1011
1101	0010
1110	0000
1111	0101

Claude Shannon and Substitution-Permutation Ciphers

- Claude Shannon introduced idea of substitution-permutation (S-P) networks in 1949 paper
- form basis of modern block ciphers
- S-P nets are based on the two primitive cryptographic operations seen before:
  - *substitution* (S-box)
  - *permutation* (P-box)
- provide *confusion* & *diffusion* of message & key

Feistel proposed that we can approximate the ideal block cipher by utilizing the concept of a product cipher, which is the execution of two or more simple ciphers in sequence in such a way that the final result or product is cryptographically stronger than any of the component ciphers. In particular, Feistel proposed the use of a cipher that alternates substitutions and permutations, as a practical application of a proposal by Claude Shannon. Claude Shannon's 1949 paper has the key ideas that led to the development of modern block ciphers. Critically, it was the technique of layering groups of S-boxes separated by a larger P-box to form the S-P network, a complex form of a product cipher. He also introduced the ideas of *confusion* and *diffusion*, notionally provided by S-boxes and P-boxes (in conjunction with S-boxes).

Substitution/ Permutation

- **Substitution:** Each plaintext element or group of elements is uniquely replaced by a corresponding ciphertext element or group of elements.
- • **Permutation:** A sequence of plaintext elements is replaced by a permutation of that sequence. That is, no elements are added or deleted or replaced in the sequence, rather the order in which the elements appear in the sequence is changed.

Confusion and Diffusion

- cipher needs to completely obscure statistical properties of original message
- a one-time pad does this
- more practically Shannon suggested combining S & P elements to obtain:
- **diffusion** – dissipates statistical structure of plaintext over bulk of ciphertext
  - An example of diffusion is to encrypt a message of characters with an averaging operation
- **confusion** – makes relationship between ciphertext and key as complex as possible

The terms diffusion and confusion were introduced by Claude Shannon to capture the two basic building blocks for any cryptographic system. Shannon's concern was to thwart cryptanalysis based on statistical analysis. Every block cipher involves a transformation of a block of plaintext into a block of ciphertext, where the transformation depends on the key. The mechanism of diffusion seeks to make the statistical relationship between the plaintext and ciphertext as complex as possible in order to thwart attempts to deduce the key. Confusion seeks to make the relationship between the statistics of the ciphertext and the value of the encryption key as complex as possible, again to thwart attempts to discover the key.

So successful are diffusion and confusion in capturing the essence of the desired attributes of a block cipher that they have become the cornerstone of modern block cipher design.

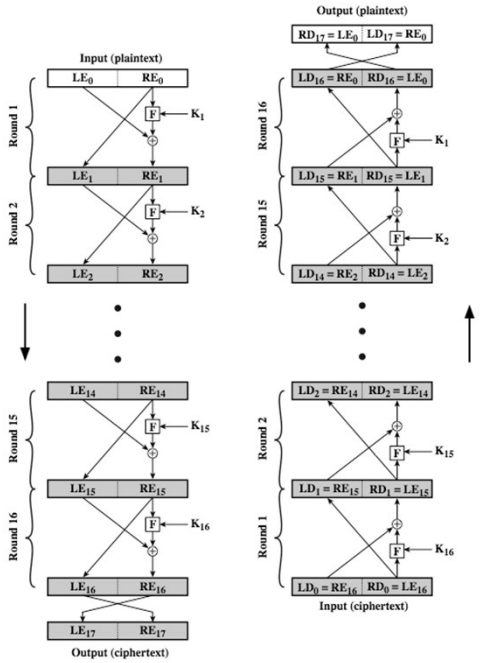
Feistel Cipher Structure

- Horst Feistel devised the **feistel cipher**
  - based on concept of invertible product cipher
- partitions input block into two halves
  - process through multiple rounds which
  - perform a substitution on left data half
  - based on round function of right half & subkey
  - then have permutation swapping halves
- implements Shannon's S-P net concept

Horst Feistel, working at IBM Thomas J Watson Research Labs devised a suitable invertible cipher structure in early 70's.

One of Feistel's main contributions was the invention of a suitable structure which adapted Shannon's S-P network in an easily inverted structure. It partitions input block into two halves which are processed through multiple rounds which perform a substitution on left data half, based on round function of right half & subkey, and then have permutation swapping halves. Essentially the same h/w or s/w is used for both encryption and decryption, with just a slight change in how the keys are used. One layer of S-boxes and the following P-box are used to form the round function.

Feistel Cipher Structure



Stallings Figure 3.3 illustrates the classical feistel cipher structure, with data split in 2 halves, processed through a number of rounds which perform a substitution on left half using output of round function on right half & key, and a permutation which swaps halves, as listed previously. The LHS side of this figure shows the flow during encryption, the RHS in decryption.

The inputs to the encryption algorithm are a plaintext block of length 2w bits and a key K. The plaintext block is divided into two halves, L_0 and R_0. The two halves of the data pass through n rounds of processing and then combine to produce the ciphertext block. Each round i has as inputs L_{i-1} and R_{i-1}, derived from the previous round, as well as a subkey K_i, derived from the overall K. In general, the subkeys K are different from K and from each other.

The process of decryption with a Feistel cipher is essentially the same as the encryption process. The rule is as follows: Use the ciphertext as input to the algorithm, but use the subkeys K_i in reverse order. That is, use K_n in the first round, K_{n-1} in the second round, and so on until K_1 is used in the last round. This is a nice feature because it means we need not implement two different algorithms, one for encryption and one for decryption. See discussion in text for why using the same algorithm with a reversed key order produces the correct result, noting that at every round, the intermediate value of the decryption process is equal to the corresponding value of the encryption process with the two halves of the value swapped.

Feistel Cipher Structure

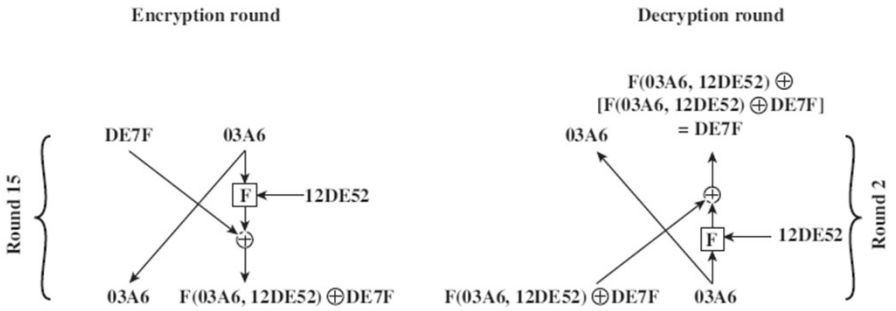


Figure 3.4 Feistel Example

The XOR has the following properties:

$$[A \oplus B] \oplus C = A \oplus [B \oplus C]$$
$$D \oplus D = 0$$
$$E \oplus 0 = E$$

Feistel Cipher Design Elements

- block size
- key size
- number of rounds
- subkey generation algorithm
- round function
- fast software en/decryption
- ease of analysis

The exact realization of a Feistel network depends on the choice of the following parameters and design features:

• block size - increasing size improves security, but slows cipher

• key size - increasing size improves security, makes exhaustive key searching harder, but may slow cipher

• number of rounds - increasing number improves security, but slows cipher

• subkey generation algorithm - greater complexity can make analysis harder, but slows cipher

• round function - greater complexity can make analysis harder, but slows cipher

• fast software en/decryption - more recent concern for practical use

• ease of analysis - for easier validation & testing of strength

Data Encryption Standard (DES)

- most widely used block cipher in world
- adopted in 1977 by NBS (now NIST)
  - as FIPS PUB 46
- encrypts 64-bit data using 56-bit key
- has widespread use
- has been considerable controversy over its security

The most widely used private key block cipher, is the Data Encryption Standard (DES). It was adopted in 1977 by the National Bureau of Standards as Federal Information Processing Standard 46 (FIPS PUB 46). DES encrypts data in 64-bit blocks using a 56-bit key. The DES enjoys widespread use. It has also been the subject of much controversy its security.

DES History

- IBM developed Lucifer cipher
  - by team led by Feistel in late 60's
  - used 64-bit data blocks with 128-bit key
- then redeveloped as a commercial cipher with input from NSA and others
- in 1973 NBS issued request for proposals for a national cipher standard
- IBM submitted their revised Lucifer which was eventually accepted as the DES

In the late 1960s, IBM set up a research project in computer cryptography led by Horst Feistel. The project concluded in 1971 with the development of the LUCIFER algorithm. LUCIFER is a Feistel block cipher that operates on blocks of 64 bits, using a key size of 128 bits.

Because of the promising results produced by the LUCIFER project, IBM embarked on an effort, headed by Walter Tuchman and Carl Meyer, to develop a marketable commercial encryption product that ideally could be implemented on a single chip. It involved not only IBM researchers but also outside consultants and technical advice from NSA. The outcome of this effort was a refined version of LUCIFER that was more resistant to cryptanalysis but that had a reduced key size of 56 bits, to fit on a single chip.

In 1973, the National Bureau of Standards (NBS) issued a request for proposals for a national cipher standard. IBM submitted the modified LUCIFER. It was by far the best algorithm proposed and was adopted in 1977 as the Data Encryption Standard.

DES Design Controversy

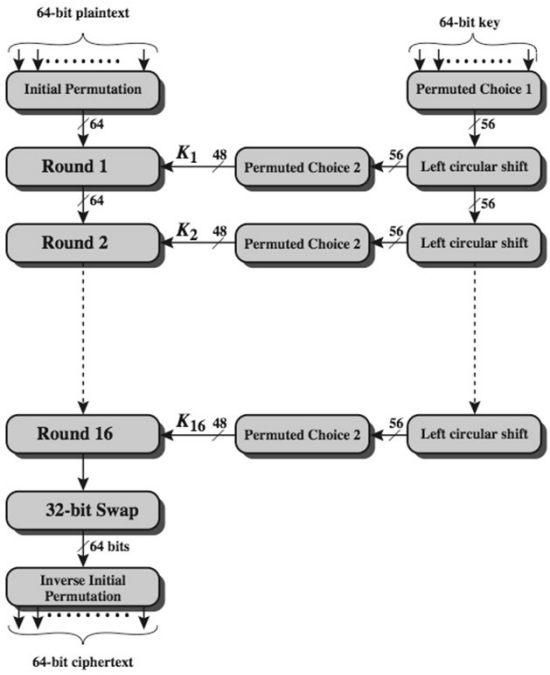
- although DES standard is public
- was considerable controversy over design
  - in choice of 56-bit key (vs Lucifer 128-bit)
  - and because design criteria were classified
- subsequent events and public analysis show in fact design was appropriate
- use of DES has flourished
  - especially in financial applications
  - still standardised for legacy application use

Before its adoption as a standard, the proposed DES was subjected to intense & continuing criticism over the size of its key & the classified design criteria.

Recent analysis has shown despite this controversy, that DES is well designed. DES is theoretically broken using Differential or Linear Cryptanalysis but in practise is unlikely to be a problem yet. Also rapid advances in computing speed though have rendered the 56 bit key susceptible to exhaustive key search, as predicted by Diffie & Hellman.

DES has flourished and is widely used, especially in financial applications. It is still standardized for legacy systems, with either AES or triple DES for new applications.

DES Encryption Overview



The overall scheme for DES encryption is illustrated in Stallings Figure 3.4, which takes as input 64-bits of data and of key.

The left side shows the basic process for enciphering a 64-bit data block which consists of:

- an initial permutation (IP) which shuffles the 64-bit input block

- 16 rounds of a complex key dependent round function involving substitutions & permutations

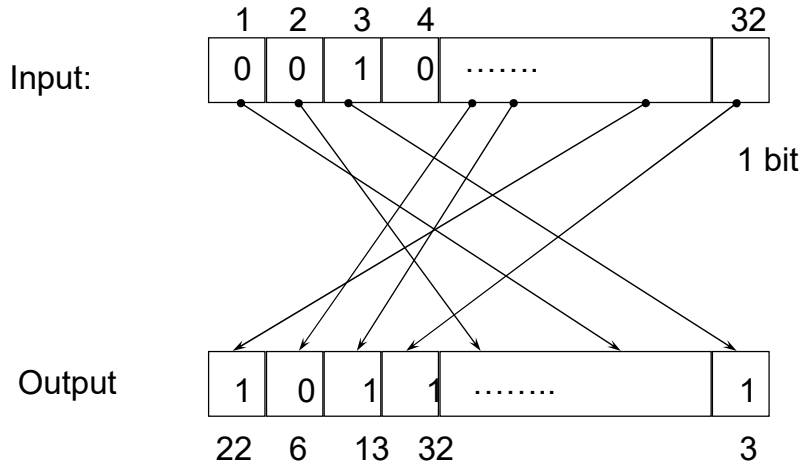
- a final permutation, being the inverse of IP

The right side shows the handling of the 56-bit key and consists of:

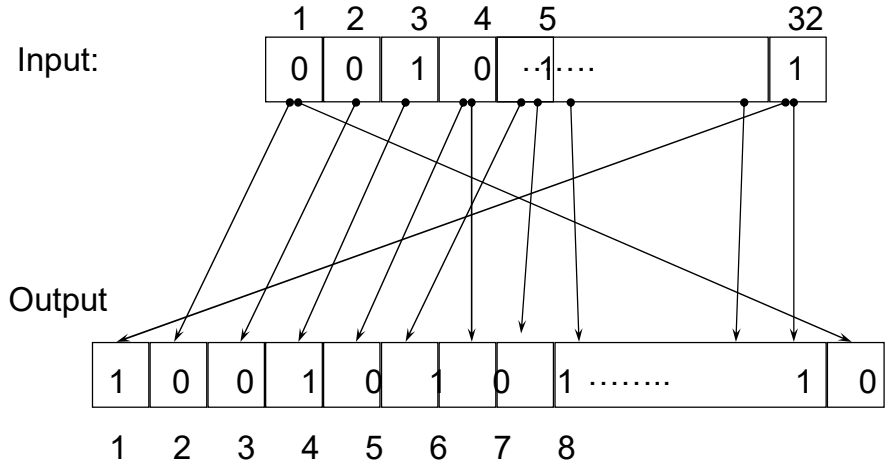
- an initial permutation of the key (PC1) which selects 56-bits out of the 64-bits input, in two 28-bit halves

- 16 stages to generate the 48-bit subkeys using a left circular shift and a permutation of the two 28-bit halves

Bit Permutation (1-to-1)



Bits Expansion (1-to-m)



Initial and Final Permutations

Table 3.2 Permutation Tables for DES

(a) Initial Permutation (IP)

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

(b) Inverse Initial Permutation (IP^{-1})

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Initial and Final Permutations

- Initial permutation (*IP*)
- View the input as M: (8X8 -bit) matrix
- Transform M into M1 in two steps
  - Transpose row *x* into column (9-*x*), 0<*x*<9
  - Apply permutation on the rows:
    - For even column *y*, it becomes row *y*/2
    - For odd column *y*, it becomes row (5+*y*/2)
- Final permutation $FP = IP^{-1}$

Initial Permutation IP

- first step of the data computation
- IP reorders the input data bits
- even bits to LH half, odd bits to RH half
- quite regular in structure
- example:

```
IP(675a6967 5e5a6b5a) = (ffb2194d 004df6fb)
```

The initial permutation and its inverse are defined by tables, as shown in Stallings Tables 3.2a and 3.2b, respectively. The tables are to be interpreted as follows. The input to a table consists of 64 bits numbered left to right from 1 to 64. The 64 entries in the permutation table contain a permutation of the numbers from 1 to 64. Each entry in the permutation table indicates the position of a numbered input bit in the output, which also consists of 64 bits.

Note that the bit numbering for DES reflects IBM mainframe practice, and is the opposite of what we now mostly use - so be careful! Numbers from Bit 1 (leftmost, most significant) to bit 32/48/64 etc (rightmost, least significant).

For example, a 64-bit plaintext value of “675a6967 5e5a6b5a” (written in left & right halves) after permuting with IP becomes “ffb2194d 004df6fb”. Note that example values are specified using hexadecimal.

DES Round Structure

- uses two 32-bit L & R halves
- as for any Feistel cipher can describe as:

  $L_i = R_{i-1}$

  $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$
- F takes 32-bit R half and 48-bit subkey:
  - expands R to 48-bits using perm E
  - adds to subkey using XOR
  - passes through 8 S-boxes to get 32-bit result
  - finally permutes using 32-bit perm P

We now review the internal structure of the DES round function F, which takes R half & subkey, and processes them. The round key K_i is 48 bits. The R input is 32 bits. This R input is first expanded to 48 bits by using a table that defines a permutation plus an expansion that involves duplication of 16 of the R bits (Table 3.2c). The resulting 48 bits are XORed with K_i This 48-bit result passes through a substitution function that produces a 32-bit output, which is permuted as defined by Table 3.2d. This follows the classic structure for a feistel cipher.

Note that the s-boxes provide the “confusion” of data and key values, whilst the permutation P then spreads this as widely as possible, so each S-box output affects as many S-box inputs in the next round as possible, giving “diffusion”.

Expansion

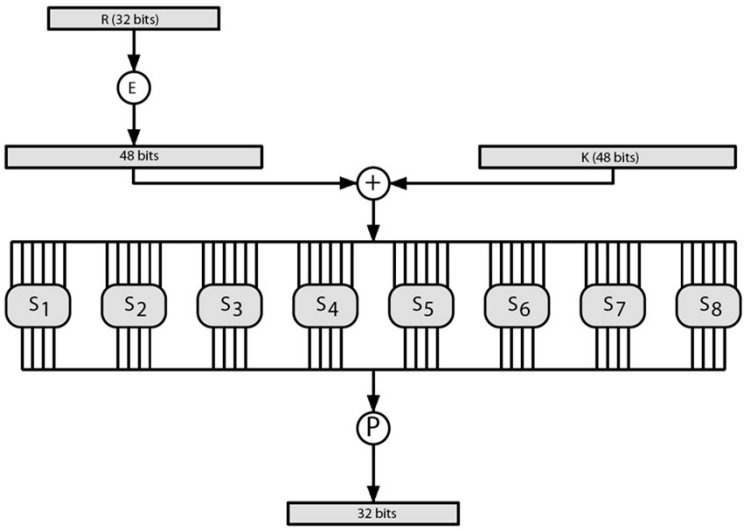
(c) Expansion Permutation (E)

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

(d) Permutation Function (P)

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

DES Round Structure



Stallings Figure 3.7 illustrates the internal structure of the DES round function F. The R input is first expanded to 48 bits by using expansion table E that defines a permutation plus an expansion that involves duplication of 16 of the R bits (Stallings Table 3.2c). The resulting 48 bits are XORed with key K_i. This 48-bit result passes through a substitution function comprising 8 S-boxes which each map 6 input bits to 4 output bits, producing a 32-bit output, which is then permuted by permutation P as defined by Stallings Table 3.2d.

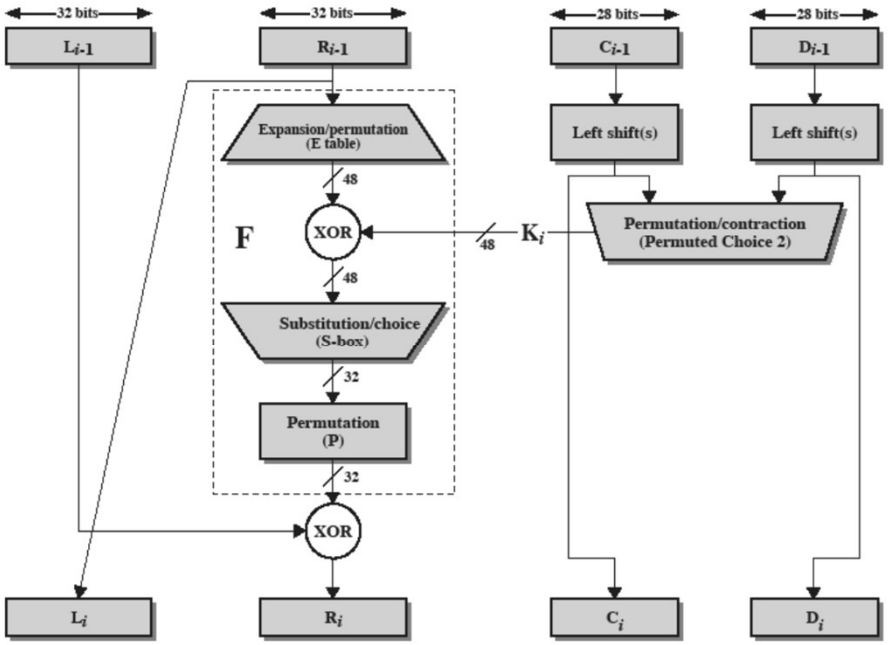
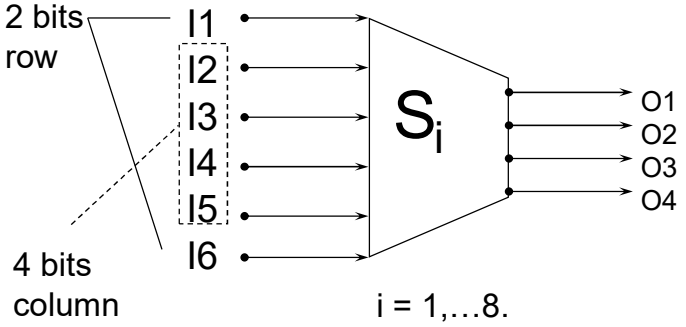


Figure 3.6 Single Round of DES Algorithm

S-Box (Substitute and Shrink)

- 48 bits ==> 32 bits. (8*6 ==> 8*4)
- 2 bits used to select amongst 4 permutations for the rest of the 4-bit quantity



Substitution Boxes S

- have eight S-boxes which map 6 to 4 bits
- each S-box is actually 4 little 4 bit boxes
  - outer bits 1 & 6 (**row** bits) select one row of 4
  - inner bits 2-5 (**col** bits) are substituted
  - result is 8 lots of 4 bits, or 32 bits
- row selection depends on both data & key
  - feature known as autoclaving (autokeying)
- example:
  - `S(18 09 12 3d 11 17 38 39) = 5fd25e03`

The substitution consists of a set of eight S-boxes, each of which accepts 6 bits as input and produces 4 bits as output. These transformations are defined in Stallings Table 3.3, which is interpreted as follows: The first and last bits of the input to box S*i* form a 2-bit binary number to select one of four substitutions defined by the four rows in the table for S*i*. The middle four bits select one of the sixteen columns. The decimal value in the cell selected by the row and column is then converted to its 4-bit representation to produce the output. For example, in S1, for input 011001, the row is 01 (row 1) and the column is 1100 (column 12). The value in row 1, column 12 is 9, so the output is 1001.

The example lists 8 6-bit values (ie 18 in hex is 011000 in binary, 09 hex is 001001 binary, 12 hex is 010010 binary, 3d hex is 111101 binary etc), each of which is replaced following the process detailed above using the appropriate S-box. ie

S1(011000) lookup row 00 col 1100 in S1 to get 5

S2(001001) lookup row 01 col 0100 in S2 to get 15 = f in hex

S3(010010) lookup row 00 col 1001 in S3 to get 13 = d in hex

S4(111101) lookup row 11 col 1110 in S4 to get 2 etc

DES Key Schedule

- forms subkeys used in each round
  - initial permutation of the key (PC1) which selects 56-bits in two 28-bit halves
  - 16 stages consisting of:
    - rotating **each half** separately either 1 or 2 places depending on the **key rotation schedule** K
    - selecting 24-bits from each half & permuting them by PC2 for use in round function F
- note practical use issues in h/w vs s/w

The DES Key Schedule generates the subkeys needed for each data encryption round. A 64-bit key is used as input to the algorithm, though every eighth bit is ignored, as indicated by the lack of shading in Table 3.4a. It is first processed by Permuted Choice One (Stallings Table 3.4b). The resulting 56-bit key is then treated as two 28-bit quantities C & D. In each round, these are separately processed through a circular left shift (rotation) of 1 or 2 bits as shown in Stallings Table 3.4d. These shifted values serve as input to the next round of the key schedule. They also serve as input to Permuted Choice Two (Stallings Table 3.4c), which produces a 48-bit output that serves as input to the round function F.

The 56 bit key size comes from security considerations as we know now. It was big enough so that an exhaustive key search was about as hard as the best direct attack (a form of differential cryptanalysis called a T-attack, known by the IBM & NSA researchers), but no bigger. The extra 8 bits were then used as parity (error detecting) bits, which makes sense given the original design use for hardware communications links. However we hit an incompatibility with simple s/w implementations since the top bit in each byte is 0 (since ASCII only uses 7 bits), but the DES key schedule throws away the bottom bit! A good implementation needs to be cleverer!

DES Decryption

- decrypt must unwind steps of data computation
- with Feistel design, do encryption steps again using subkeys in reverse order (SK16 … SK1)
  - IP undoes final FP step of encryption
  - 1st round with SK16 undoes 16th encrypt round
  - ….
  - 16th round with SK1 undoes 1st encrypt round
  - then final FP undoes initial encryption IP
  - thus recovering original data value

As with any Feistel cipher, DES decryption uses the same algorithm as encryption except that the subkeys are used in reverse order SK16 .. SK1.

If you trace through the DES overview diagram can see how each decryption step top to bottom with reversed subkeys, undoes the equivalent encryption step moving from bottom to top.

DES Example

Round	K_i	L_i	R_i
IP		5a005a00	3cf03c0f
1	1e030f03080d2930	3cf03c0f	bad22845
2	0a31293432242318	bad22845	99e9b723
3	23072318201d0c1d	99e9b723	0bae3b9e
4	05261d3824311a20	0bae3b9e	42415649
5	3325340136002c25	42415649	18b3fa41
6	123a2d0d04262a1c	18b3fa41	9616fe23
7	021f120b1c130611	9616fe23	67117cf2
8	1c10372a2832002b	67117cf2	c11bfc09
9	04292a380c341f03	c11bfc09	887fbc6c
10	2703212607280403	887fbc6c	600f7e8b
11	2826390c31261504	600f7e8b	f596506e
12	12071c241a0a0f08	f596506e	738538b8
13	300935393c0d100b	738538b8	c6a62c4e
14	311e09231321182a	c6a62c4e	56b0bd75
15	283d3e0227072528	56b0bd75	75e8fd8f
16	2921080b13143025	75e8fd8f	25896490
IP^{-1}		da02ce3a	89ecac3b

Can now work through an example, and consider some of its implications. In this example, the plaintext is a hexadecimal palindrome, with:

Plaintext: 02468aceeca86420

Key: 0f1571c947d9e859

Ciphertext: da02ce3a89ecac3b

Table 3.5 shows the progression of the algorithm. The first row shows the 32-bit values of the left and right halves of data after the initial permutation. The next 16 rows show the results after each round. Also shown is the value of the 48-bit subkey generated for each round. The final row shows the left and right-hand values after the inverse initial permutation. These two values combined form the ciphertext.

Avalanche in DES

Round		δ	Round		δ
	02468aceeca86420 12468aceeca86420	1	9	c11bfc09887fbc6c 99f911532eed7d94	32
1	3cf03c0fbad22845 3cf03c0fbad32845	1	10	887fbc6c600f7e8b 2eed7d94d0f23094	34
2	bad2284599e9b723 bad3284539a9b7a3	5	11	600f7e8bf596506e d0f23094455da9c4	37
3	99e9b7230bae3b9e 39a9b7a3171cb8b3	18	12	f596506e738538b8 455da9c47f6e3cf3	31
4	0bae3b9e42415649 171cb8b3ccaca55e	34	13	738538b8c6a62c4e 7f6e3cf34bc1a8d9	29
5	4241564918b3fa41 ccaca55ed16c3653	37	14	c6a62c4e56b0bd75 4bc1a8d91e07d409	33
6	18b3fa419616fe23 d16c3653cf402c68	33	15	56b0bd7575e8fd8f 1e07d4091ce2e6dc	31
7	9616fe2367117cf2 cf402c682b2cefbc	32	16	75e8fd8f25896490 1ce2e6dc365e5f59	32
8	67117cf2c11bfc09 2b2cefbc99f91153	33	IP^{-1}	da02ce3a89ecac3b 057cde97d7683f2a	32

A desirable property of any encryption algorithm is that a small change in either the plaintext or the key should produce a significant change in the ciphertext. In particular, a change in one bit of the plaintext or one bit of the key should produce a change in many bits of the ciphertext. This is referred to as the avalanche effect. Using the example from Table 3.5, Table 3.6 shows the result when the fourth bit of the plaintext is changed, so that the plaintext is 12468aceeca86420. The second column of the table shows the intermediate 64-bit values at the end of each round for the two plaintexts. The third column shows the number of bits that differ between the two intermediate values. The table shows that after just three rounds, 18 bits differ between the two blocks. On completion, the two ciphertexts differ in 32 bit positions. Table 3.7 in the text shows a similar test using the original plaintext of with two keys that differ in only the fourth bit position. Again, the results show that about half of the bits in the ciphertext differ and that the avalanche effect is pronounced after just a few rounds.

Avalanche Effect

- key desirable property of encryption alg
- where a change of **one** input or key bit results in changing approx **half** output bits
- making attempts to "home-in" by guessing keys impossible
- DES exhibits strong avalanche

A desirable property of any encryption algorithm is that a small change in either the plaintext or the key should produce a significant change in the ciphertext. In particular, a change in one bit of the plaintext or one bit of the key should produce a change in many bits of the ciphertext. If the change were small, this might provide a way to reduce the size of the plaintext or key space to be searched. DES exhibits a strong avalanche effect, as may be seen in Stallings Table 3.5.

Strength of DES – Key Size

- 56-bit keys have $2^{56} = 7.2 \times 10^{16}$ values
- brute force search looks hard
- recent advances have shown is possible
  - in 1997 on Internet in a few months
  - in 1998 on dedicated h/w (EFF) in a few days
  - in 1999 above combined in 22hrs!
- still must be able to recognize plaintext
- must now consider alternatives to DES

Since its adoption as a federal standard, there have been lingering concerns about the level of security provided by DES in two areas: key size and the nature of the algorithm.

With a key length of 56 bits, there are 2^{56} possible keys, which is approximately $7.2*10^{16}$ keys. Thus a brute-force attack appeared impractical.

However DES was finally and definitively proved insecure in July 1998, when the Electronic Frontier Foundation (EFF) announced that it had broken a DES encryption using a special-purpose "DES cracker" machine that was built for less than $250,000. The attack took less than three days. The EFF has published a detailed description of the machine, enabling others to build their own cracker [EFF98].

There have been other demonstrated breaks of the DES using both large networks of computers & dedicated h/w, including:

- 1997 on a large network of computers in a few months

- 1998 on dedicated h/w (EFF) in a few days

- 1999 above combined in 22hrs!

It is important to note that there is more to a key-search attack than simply running through all possible keys. Unless known plaintext is provided, the analyst must be able to recognize plaintext as plaintext.

Clearly must now consider alternatives to DES, the most important of which are AES and triple DES.

Strength of DES – Analytic Attacks

- now have several analytic attacks on DES
- these utilise some deep structure of the cipher
  - by gathering information about encryptions
  - can eventually recover some/all of the sub-key bits
  - if necessary then exhaustively search for the rest
- generally these are statistical attacks
  - differential cryptanalysis
  - linear cryptanalysis
  - related key attacks

Another concern is the possibility that cryptanalysis is possible by exploiting the characteristics of the DES algorithm. The focus of concern has been on the eight substitution tables, or S-boxes, that are used in each iteration. These techniques utilise some deep structure of the cipher by gathering information about encryptions so that eventually you can recover some/all of the sub-key bits, and then exhaustively search for the rest if necessary. Generally these are statistical attacks which depend on the amount of information gathered for their likelihood of success. Attacks of this form include differential cryptanalysis. linear cryptanalysis, and related key attacks.

Strength of DES – Timing Attacks

- attacks actual implementation of cipher
- use knowledge of consequences of implementation to derive information about some/all subkey bits
- specifically use fact that calculations can take varying times depending on the value of the inputs to it
- particularly problematic on smartcards

We will discuss timing attacks in more detail later, as they relate to public-key algorithms. However, the issue may also be relevant for symmetric ciphers. A timing attack is one in which information about the key or the plaintext is obtained by observing how long it takes a given implementation to perform decryptions on various ciphertexts. A timing attack exploits the fact that an encryption or decryption algorithm often takes slightly different amounts of time on different inputs. The AES analysis process has highlighted this attack approach, and showed that it is a concern particularly with smartcard implementations, though DES appears to be fairly resistant to a successful timing attack.

Differential Cryptanalysis

- one of the most significant recent (public) advances in cryptanalysis
- known by NSA in 70's cf DES design
- Murphy, Biham & Shamir published in 90's
- powerful method to analyse block ciphers
- used to analyse most current block ciphers with varying degrees of success
- DES reasonably resistant to it, cf Lucifer

Biham & Shamir show Differential Cryptanalysis can be successfully used to cryptanalyse the DES with an effort on the order of 2^{47} encryptions, requiring 2^{47} chosen plaintexts. Although 2^{47} is certainly significantly less than 2^{55}, the need for the adversary to find 2^{47} chosen plaintexts makes this attack of only theoretical interest. They also demonstrated this form of attack on a variety of encryption algorithms and hash functions.

Differential cryptanalysis was known to the IBM DES design team as early as 1974 (as a T attack), and influenced the design of the S-boxes and the permutation P to improve its resistance to it. Compare DES's security with the cryptanalysis of an eight-round LUCIFER algorithm which requires only 256 chosen plaintexts, verses an attack on an eight-round version of DES requires 2^{14} chosen plaintexts.

Differential Cryptanalysis

- a statistical attack against Feistel ciphers
- uses cipher structure not previously used
- design of S-P networks has output of function *f* influenced by both input & key
- hence cannot trace values back through cipher without knowing value of the key
- differential cryptanalysis compares two related pairs of encryptions

The differential cryptanalysis attack is complex. The rationale behind differential cryptanalysis is to observe the behavior of pairs of text blocks evolving along each round of the cipher, instead of observing the evolution of a single text block. Each round of DES maps the right-hand input into the left-hand output and sets the right-hand output to be a function of the left-hand input and the subkey for this round, which means you cannot trace values back through cipher without knowing the value of the key. Differential Cryptanalysis compares two related pairs of encryptions, which can leak information about the key, given a sufficiently large number of suitable pairs.

Differential Cryptanalysis Compares Pairs of Encryptions

- with a known difference in the input
- searching for a known difference in output
- when same subkeys are used

$$\begin{aligned}\Delta m_{i+1} &= m_{i+1} \oplus m'_{i+1} \\ &= [m_{i-1} \oplus \mathrm{f}(m_i, K_i)] \oplus [m'_{i-1} \oplus \mathrm{f}(m'_i, K_i)] \\ &= \Delta m_{i-1} \oplus [\mathrm{f}(m_i, K_i) \oplus \mathrm{f}(m'_i, K_i)]\end{aligned}$$

This attack is known as **Differential Cryptanalysis** because the analysis compares **differences** between two related encryptions, and looks for a **known difference in** leading to a **known difference out** with some (pretty small but still significant) probability. If a number of such differences are determined, it is feasible to determine the subkey used in the function f.

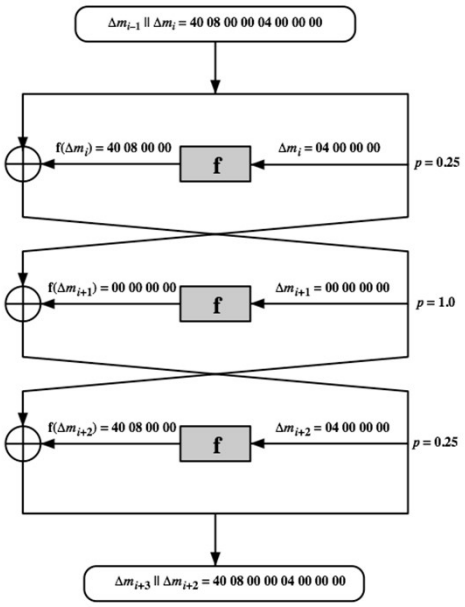
In differential cryptanalysis, we start with two messages, *m* and *m'*, with a known XOR difference dm = m xor m', and consider the difference between the intermediate message halves: dm = m xor m'. Then we have the equation from Stallings section 3.4 which shows how this removes the influence of the key, hence enabling the analysis. Suppose that many pairs of inputs to f with the same difference yield the same output difference if the same subkey is used. To put this more precisely, let us say that X may cause Y with probability p, if for a fraction p of the pairs in which the input XOR is X, the output XOR equals Y. We want to suppose that there are a number of values of X that have high probability of causing a particular output difference.

Differential Cryptanalysis

- have some input difference giving some output difference with probability p
- if find instances of some higher probability input / output difference pairs occurring
- can infer subkey that was used in round
- then must iterate process over many rounds (with decreasing probabilities)

The overall strategy of differential cryptanalysis is based on these considerations for a single round. The procedure is to begin with two plaintext messages m and m' with a given difference and trace through a probable pattern of differences after each round to yield a probable difference for the ciphertext. You submit m and m' for encryption to determine the actual difference under the unknown key and compare the result to the probable difference. If there is a match, then suspect that all the probable patterns at all the intermediate rounds are correct. With that assumption, can make some deductions about the key bits. This procedure must be repeated many times to determine all the key bits.

Differential Cryptanalysis



Stallings Figure 3.7 illustrates the propagation of differences through three rounds of DES. The probabilities shown on the right refer to the probability that a given set of intermediate differences will appear as a function of the input differences. Overall, after three rounds the probability that the output difference is as shown is equal to 0.25*1*0.25=0.0625. Since the output difference is the same as the input, this 3 round pattern can be iterated over a larger number of rounds, with probabilities multiplying to be successively smaller.

Differential Cryptanalysis

- perform attack by repeatedly encrypting plaintext pairs with known input XOR until obtain desired output XOR
- when found
  - if intermediate rounds match required XOR have a **right pair**
  - if not then have a **wrong pair**, relative ratio is S/N for attack
- can then deduce keys values for the rounds
  - right pairs suggest same key bits
  - wrong pairs give random values
- for large numbers of rounds, probability is so low that more pairs are required than exist with 64-bit inputs
- Biham and Shamir have shown how a 13-round

Differential Cryptanalysis works by performing the attack by repeatedly encrypting plaintext pairs with known input XOR until obtain desired output XOR. See [BIHA93] for detailed descriptions. Attack on full DES requires an effort on the order of 2^{47} encryptions, requiring 2^{47} chosen plaintexts to be encrypted, with a considerable amount of analysis – in practise exhaustive search is still easier, even though up to 2^{55} encryptions are required for this.

Linear Cryptanalysis

- another recent development
- also a statistical method
- must be iterated over rounds, with decreasing probabilities
- developed by Matsui et al in early 90's
- based on finding linear approximations
- can attack DES with 2^{43} known plaintexts, easier but still in practise infeasible

A more recent development is linear cryptanalysis. This attack is based on finding linear approximations to describe the transformations performed in DES. This method can find a DES key given 2^43 known plaintexts, as compared to 2^47 chosen plaintexts for differential cryptanalysis. Although this is a minor improvement, because it may be easier to acquire known plaintext rather than chosen plaintext, it still leaves linear cryptanalysis infeasible as an attack on DES. Again, this attack uses structure not seen before. So far, little work has been done by other groups to validate the linear cryptanalytic approach.

Linear Cryptanalysis

- find linear approximations with prob p != $\frac{1}{2}$

  $P[i_1,i_2,...,i_a] \oplus C[j_1,j_2,...,j_b] = K[k_1,k_2,...,k_c]$

  where i_a,j_b,k_c are bit locations in P,C,K
- gives linear equation for key bits
- get one key bit using max likelihood alg
- using a large number of trial encryptions
- effectiveness given by: $|p-1/2|$

The objective of linear cryptanalysis is to find an effective linear equation relating some plaintext, ciphertext and key bits that holds with probability p<>0.5 as shown. Once a proposed relation is determined, the procedure is to compute the results of the left-hand side of the equation for a large number of plaintext-ciphertext pairs, in order to determine whether the sum of the key bits is 0 or 1, thus giving 1 bit of info about them. This is repeated for other equations and many pairs to derive some of the key bit values. Because we are dealing with linear equations, the problem can be approached one round of the cipher at a time, with the results combined. See [MATS93] for details.

DES Design Criteria

- as reported by Coppersmith in [COPP94]
- 7 criteria for S-boxes provide for
  - non-linearity
  - resistance to differential cryptanalysis
  - good confusion
- 3 criteria for permutation P provide for
  - increased diffusion

Although much progress has been made in designing block ciphers that are cryptographically strong, the basic principles have not changed all that much since the work of Feistel and the DES design team in the early 1970s. Some of the criteria used in the design of DES were reported in [COPP94], and focused on the design of the S-boxes and on the P function that distributes the output of the S boxes, as summarized above. See text for further details.

Block Cipher Design

- basic principles still like Feistel's in 1970's
- number of rounds
  - more is better, exhaustive search best attack
- function f:
  - provides "confusion", is nonlinear, avalanche
  - have issues of how S-boxes are selected
- key schedule
  - complex subkey creation, key avalanche

The cryptographic strength of a Feistel cipher derives from three aspects of the design: the number of rounds, the function F, and the key schedule algorithm. Briefly discuss these.

The greater the number of rounds, the more difficult it is to perform cryptanalysis, even for a relatively weak F. In general, the criterion should be that the number of rounds is chosen so that known cryptanalytic efforts require greater effort than a simple brute-force key search attack. This criterion is attractive because it makes it easy to judge the strength of an algorithm and to compare different algorithms.

The function F provides the element of confusion in a Feistel cipher, want it to be difficult to "unscramble" the substitution performed by F. One obvious criterion is that F be nonlinear. The more nonlinear F, the more difficult any type of cryptanalysis will be. We would like it to have good avalanche properties, or even the strict avalanche criterion (SAC). Another criterion is the bit independence criterion (BIC). One of the most intense areas of research in the field of symmetric block ciphers is that of S-box design. Would like any change to the input vector to an S-box to result in random-looking changes to the output. The relationship should be nonlinear and difficult to approximate with linear functions.

A final area of block cipher design, and one that has received less attention than S-box design, is the key schedule algorithm. With any Feistel block cipher, the key schedule is used to generate a subkey for each round. Would like to select subkeys to maximize the difficulty of deducing individual subkeys and the difficulty of working back to the main key. The key schedule should guarantee key/ciphertext Strict Avalanche Criterion and Bit Independence Criterion.

Summary

- have considered:
  - block vs stream ciphers
  - Feistel cipher design & structure
  - DES
    - details
    - strength
  - Differential & Linear Cryptanalysis
  - block cipher design principles

Chapter 3 summary.